



Déploiement en production d'un modèle de Machine Learning

Projet 5

Prédiction de l'attrition des salariés

Kseniia Sazonova

FastAPI

Cloud SQL PostgreSQL

Cloud Run

CI/CD

GCP

01 Contexte & mission

02 Le modèle : usage & résultats

03 Architecture & choix techniques

04 Dépôt Git & CI/CD

05 API & documentation Swagger

06 Tests & couverture

07 Base de données & modèle de données

08 Sécurité & déploiement

09 Démonstration & bilan

Contexte & mission

- ▶ Client : Futurisys souhaite rendre ses modèles ML opérationnels via une API
- ▶ Exigences : fiabilité (tests), versionnage du code (Git), déploiement
- ▶ Modèle : le classifieur d'attrition du projet 4
- ▶ Livrable : un Proof of Concept fonctionnel, déployé et documenté

Le modèle : usage et résultats

0,89

rappel (départs détectés)

47

variables

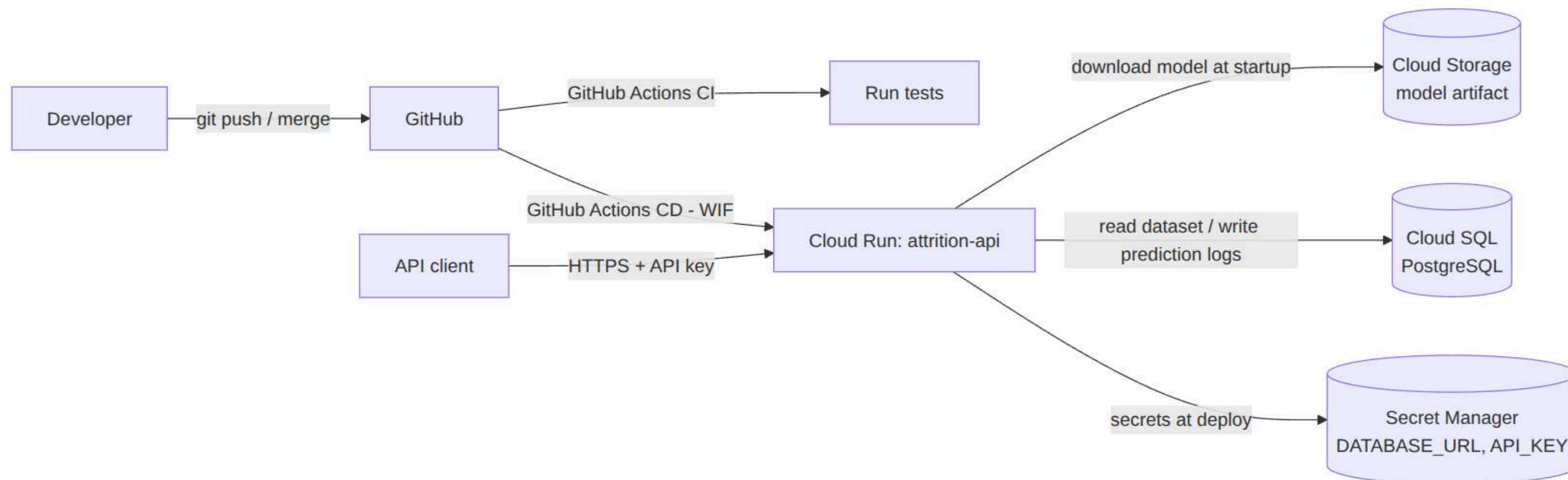
1 470

salariés

- ▶ Cas d'usage : outil d'aide à la décision RH pour cibler la rétention
- ▶ Gradient Boosting, données déséquilibrées (84 % / 16 %)
- ▶ Seuil abaissé à 0,07 → priorité au rappel (rater un départ coûte cher)

Vue d'ensemble de l'architecture

- ▶ **API** : FastAPI sur Cloud Run
- ▶ **Modèle** : Cloud Storage · **Données** : Cloud SQL (PostgreSQL)
- ▶ **Secrets** : Secret Manager · **CI/CD** : GitHub Actions
- ▶ Tout sur Google Cloud (europe-west1), sans machine locale en production



Dépôt Git & versionnage

- ▶ Structure claire : séparation app / scripts / tests / docs
- ▶ Branches dédiées par fonctionnalité (feature/*) + Pull Requests
- ▶ Tags de versionnage sémantique : v0.1.0 → v1.0.1
- ▶ Commits conventionnels (feat, test, ci, docs) — historique lisible
- ▶ README complet + documentation technique du modèle

```
attrition-classification/  
├─ app/                # FastAPI application  
│   ├── main.py        # endpoints + startup (model load, prediction logging)  
│   ├── model.py       # model loading (local or from GCS) and inference  
│   ├── preprocessing.py # cleaning, encoding, feature engineering  
│   ├── schemas.py     # Pydantic request/response models  
│   ├── auth.py        # API-key authentication  
│   └─ database.py     # SQLAlchemy engine + ORM models  
├─ scripts/            #  
│   ├── prepare_data.py # bronze CSVs -> silver dataset (join + preprocess)  
│   ├── train.py        # train the Gradient Boosting model  
│   └─ create_db.py     # create tables + seed the dataset into PostgreSQL  
├─ tests/              # unit + functional tests  
├─ docs/MODEL.md       # model technical documentation  
├─ .github/workflows/  # ci.yml (tests) + cd.yml (deploy)  
├─ Dockerfile  
└─ pyproject.toml
```

Merge pull request [#6](#) from ash0nyx/feature/cicd 

 ash0nyx authored last week ·  3 / 3

```
attrition-classification/  
├─ app/      API + modèle + BDD  
├─ scripts/  prépa · train · create_db  
├─ tests/    42 tests  
├─ docs/     model card  
├─ .github/  ci.yml · cd.yml  
└─ Dockerfile
```


Pipeline d'intégration & déploiement continu

- ▶ CI (ci.yml) : à chaque push / PR → lance les tests
- ▶ CD (cd.yml) : à chaque merge sur main → tests puis déploiement automatique
- ▶ Environnements : PR/branches = test · main = production
- ▶ Authentification GitHub → GCP sans clé (Workload Identity Federation)

ci.yml

on: pull_request

✓ Run tests 25s

cd.yml

on: push

✓ test 24s — ✓ deploy 4m 15s

L'API (FastAPI) & Swagger

- ▶ 4 endpoints : /health, /model/info, /predict, /predictions
- ▶ Validation des entrées avec Pydantic (types, plages, catégories)
- ▶ Documentation Swagger / OpenAPI générée automatiquement
- ▶ Entrée invalide → 422 · clé d'API manquante → 401

Attrition Classification API 0.1.0 OAS 3.1

[/openapi.json](#)

Predicts employee attrition risk using a Gradient Boosting model. Threshold is set to 0.07 to maximize recall on an imbalanced dataset.

Authorize 

Monitoring

GET **/health** Health Check 

GET **/model/info** Model Info  

Prediction

POST **/predict** Predict  

GET **/predictions** List Predictions  

Tests & couverture

42

tests pytest

~90 %

couverture

0

dépendance externe

- ▶ Unitaires + fonctionnels : prétraitement, validation, modèle, endpoints, BDD
- ▶ Intégrité de la base (contraintes) + scénarios d'erreur (401, 422, 503, 500)
- ▶ SQLite en mémoire + modèle mocké → les tests tournent partout, en CI

Coverage report: 93%

Files

Functions

Classes

coverage.py v7.15.0, created at 2026-07-03 17:46 +0200

File ▲	statements	missing	excluded	coverage
app/database.py	81	4	0	95%
app/main.py	50	12	0	76%
app/model.py	28	1	0	96%
app/preprocessing.py	31	0	0	100%
app/schemas.py	45	0	0	100%
Total	235	17	0	93%

coverage.py v7.15.0, created at 2026-07-03 17:46 +0200

Base PostgreSQL & traçabilité

- ▶ Script Python create_db.py : crée le schéma et insère 1 470 lignes
- ▶ Exemples d'entrées en base : dataset de référence + logs de prédiction
- ▶ Traçabilité complète : chaque /predict enregistre entrée ET sortie
- ▶ SQLAlchemy 2.0 + psycopg 3 · contraintes CHECK / NOT NULL

Curl

```
curl -X 'GET' \
  'https://attrition-api-716422774465.europe-west1.run.app/predictions?limit=10' \
  -H 'accept: application/json' \
  -H 'X-API-Key: 108f433a6b01958ef397291eef56195aab464251a6470c5f383a627a7250d4c4'
```

Request URL

```
https://attrition-api-716422774465.europe-west1.run.app/predictions?limit=10
```

Server response

Code

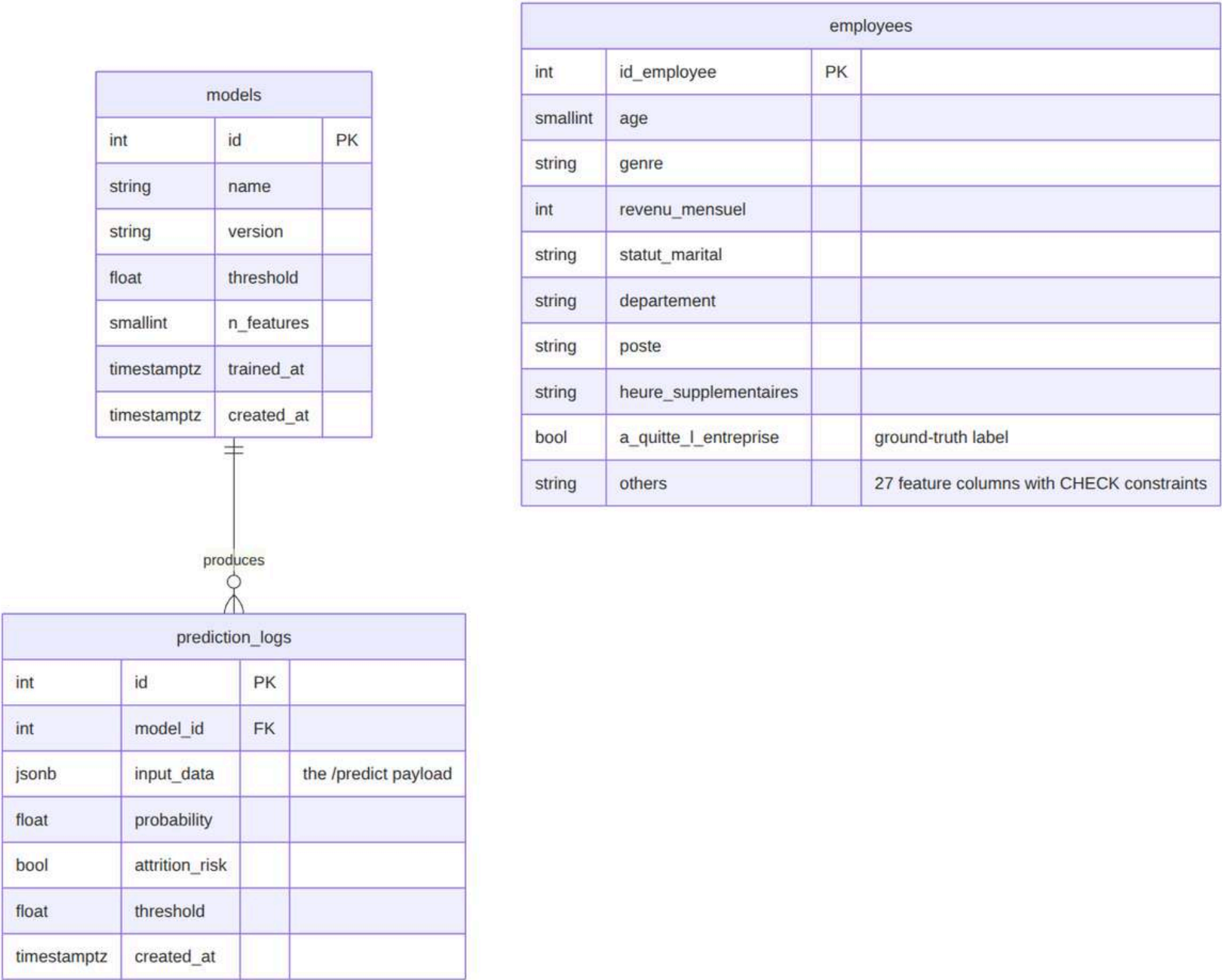
Details

200

Response body

```
{
  "probability": 0.0454,
  "attrition_risk": false,
  "threshold": 0.07
},
{
  "id": 1,
  "created_at": "2026-07-03T09:55:11.824963Z",
  "model_id": 1,
  "input_data": {
    "age": 35,
    "genre": "F",
    "poste": "Consultant",
    "departement": "Consulting",
    "domaine_etude": "Infra & Cloud",
    "revenu_mensuel": 5000,
    "statut_marital": "Marié(e)",
    "niveau_education": 3,
    "frequence_deplacement": "Occasionnel",
    "heure_supplementaires": "Non",
    "nb_formations_suivies": 2,
    "annee_experience_totale": 10,
    "annees_dans_l_entreprise": 5,
    "nombre_participation_pee": 1,
    "note_evaluation_actuelle": 3,
    "distance_domicile_travail": 10,
    "niveau_hierarchique_poste": 2,
    "note_evaluation_precedente": 3,
  }
}
```

Schéma de la base (modèle de données)



Sécurité & gestion des secrets

- ▶ Authentification par clé d'API (en-tête X-API-Key)
- ▶ Secrets (DATABASE_URL, API_KEY) dans Secret Manager, jamais dans Git
- ▶ CI/CD sans clé : Workload Identity Federation, restreint au dépôt
- ▶ Moindre privilège : compte de service dédié, rôle BDD non-superadmin

Available authorizations

APIKeyHeader (apiKey)

Name: X-API-Key

In: header

Value:

Authorize

Close

Secret Manager

[Parameter Manager](#)

[Secrets](#)

[Regional secrets](#)

[Logs](#)

Secret Manager lets you store, manage, and search secrets. [Learn more](#)

Secrets

[+ Create secret](#)

[▶ T](#)

[Filter](#) Enter property name or value

☐	Name ↑	Location
☐	API_KEY	europe-west1
☐	DATABASE_URL	europe-west1

Déploiement sur Cloud Run

- Image construite dans le cloud par Cloud Build (pas de Docker en local)
- Conteneur sur Cloud Run, connecté à Cloud SQL via socket sécurisé
- Modèle téléchargé depuis Cloud Storage au démarrage
- Versionnage sémantique des versions déployées

Revisions [Manage traffic](#)

Filter Filter revisions

	Name	Traffic	Deployed ↓
✓	attrition-api-00005-8vd	100% (to latest)	16 minutes ago
✓	attrition-api-00004-zkk	0%	21 hours ago
✓	attrition-api-00003-8xz	0%	22 hours ago
✓	attrition-api-00002-n6z	0%	22 hours ago
✓	attrition-api-00001-42p	0%	1 day ago

attrition-api Region: europe-west1 URL: <https://attrition-api-716422774465.europe-west1.run.app> Scaling:

Observability Revisions Source Triggers Networking Security YAML

Source Base image: — Edit source Show p

.dockerignore

.github

workflows

cd.yml

ci.yml

app

auth.py

database.py

```
1 # Employee Attrition Classification API
2
3 A production-deployed machine-learning API that predicts the ri
4 leaving the company. Built with FastAPI, backed by PostgreSQL f
5 and deployed to Google Cloud Run through an automated CI/CD pip
6
7 The model classifies each employee as **at risk of attrition**
8 HR, evaluation, and survey data. It is tuned to prioritise **re
9 many at-risk employees as possible, since missing a leaver is f
10 false alarm.
11
12 - **Live API:** https://attrition-api-716422774465.europe-west1
13 - **Interactive docs (Swagger):** https://attrition-api-7164227
14 - **Model card:** [docs/MODEL.md](docs/MODEL.md)
15
16 ---
17
18 ## Architecture
19
20 ```mermaid
21 flowchart LR
22     dev[Developer] -->|git push / merge| gh[GitHub]
```

Démonstration en direct

- ▶ Ouvrir Swagger : ...run.app/docs
- ▶ /health → model_loaded: true
- ▶ « Authorize » avec la clé d'API → POST /predict → une prédiction
- ▶ GET /predictions → la prédiction enregistrée en base

Difficultés & pistes d'amélioration

- ▶ Passage POC → production :
 - Migrations versionnées (Alembic) au lieu du drop/create
 - Chargement des données via un Cloud Run Job plutôt qu'en local
- ▶ Monitoring : exploiter prediction_logs pour détecter la dérive du modèle
- ▶ Authentification : passer à OAuth / IAM pour un usage réel

Bilan

- ▶ Dépôt Git structuré, branches, tags · pipeline CI/CD automatisé
- ▶ API FastAPI documentée (Swagger) et déployée sur Cloud Run
- ▶ Base PostgreSQL avec traçabilité complète des prédictions
- ▶ Tests + couverture ~90 % · sécurité (clé d'API, secrets, WIF)
- ▶ → Un POC fonctionnel, déployé en production, prêt à évoluer



Merci

Kseniia Sazonova